
Finite-Support Convolutional Likelihoods for Aggregate-Supervised Multiple-Instance Learning

Anonymous Author(s)

Affiliation

Address

email

Abstract

Weakly supervised multiple-instance learning often observes only aggregate labels for a bag, while instance labels remain hidden. Existing count-based objectives give exact likelihoods for Bernoulli counts, but many aggregate labels have richer finite structure, such as signed counts, integer-valued sums, and class histograms. We formulate aggregate-supervised Count-MIL as convolution of instance-level finite-support probability mass functions. The resulting bag likelihood recovers the standard Bernoulli count probability as a special case and extends without changing the backbone model to signed and integer-valued aggregate labels. We also derive count-conditioned posterior marginals for instance recovery and implement exact aggregate inference with GPU-parallel convolution, including an FFT-tree backend for large one-vs-rest histogram likelihoods. Experiments on binary counts, signed counts, digit sums, SVHN/UltraMNIST sum benchmarks, posterior recovery, and histogram LLP settings show that exact finite-support likelihoods can recover accurate hidden-instance predictors from aggregate labels. On CIFAR-10, ImageNet-pretrained ResNet-18 features make the one-vs-rest count likelihood competitive with LLP-PVC reference results, while a fixed-instance-budget comparison shows that the large-bag improvement is explained by greater sampled image exposure.

1 Introduction

Multiple-instance learning (MIL) studies settings where supervision is observed for a bag while the labels of individual instances remain latent. Classical binary MIL uses a positive bag label to indicate that at least one instance is positive. Many weak-supervision settings provide more structured aggregate labels: the number of positive instances, a class histogram, a signed count, or a scalar integer-valued sum. The central problem is to learn an instance-level predictor from such aggregate observations without using instance labels during training.

This paper studies aggregate-supervised MIL through finite-support likelihoods. Each instance x_i is mapped to a probability mass function over a finite set of integer-valued contributions. The observed bag label is an aggregate value, so the bag likelihood is the coefficient of that value in the convolution of the instance-level atomic PMFs. For Bernoulli contributions this recovers the standard count probability used in prior count-loss work. The same construction also gives exact likelihoods for signed counts, digit-sum labels, and one-vs-rest class-count objectives. We refer to this as atomic PMF generality: the atomic support size and contribution values define the weak-label problem, while the convolutional aggregator is unchanged. The interface is deliberately narrow: a backbone model outputs instance-level scores, and the aggregate layer converts these scores into finite-support atomic PMFs before computing the bag likelihood. The method is therefore not tied to a particular architecture. Changing the weak label changes the contribution map and convolution support, not the backbone.

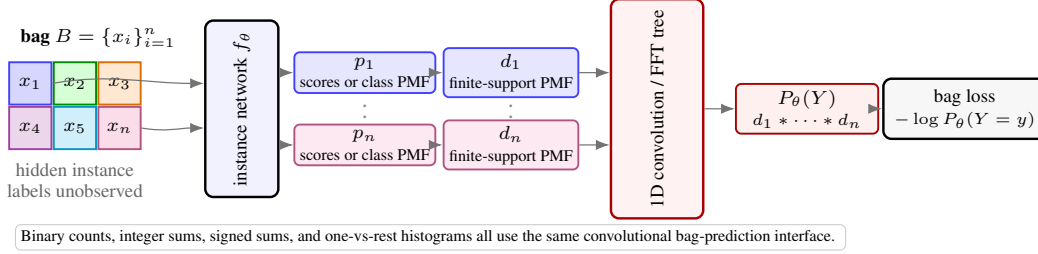


Figure 1: Convolutional aggregate pipeline. The instance network emits per-instance scores or class probabilities, which define finite-support atomic PMFs d_i . A deterministic convolutional layer computes the aggregate PMF $P_\theta(Y)$, and the observed bag label selects the likelihood term used for training.

38 This view also clarifies the boundary with prior work. Binary Bernoulli counts recover exist-
 39 ing count-loss objectives, and one-vs-rest class-count likelihoods are closely related to the count-
 40 likelihood component of LLP-PVC. Our focus is the finite-support aggregate layer that covers these
 41 cases while also extending to signed and integer-valued sum labels.

42 Our contributions are:

- 43 • We formulate aggregate-supervised Count-MIL as finite-support convolution of instance
 44 contribution distributions, explicitly recovering Bernoulli count loss as a special case rather
 45 than claiming it as new.
- 46 • We show that the same backbone-agnostic likelihood layer extends to signed counts, digit-
 47 sum labels, and one-vs-rest histogram class counts by changing only the finite-support
 48 contribution map.
- 49 • We derive count-conditioned posterior marginals for instance recovery and uncertainty di-
 50 agnostics under binary count supervision.
- 51 • We implement practical convolutional backends, including grouped signed convolution and
 52 a GPU FFT-tree backend that parallelizes large multiclass one-vs-rest PMFs beyond CPU
 53 dynamic programming.
- 54 • We evaluate hidden-instance recovery across binary, signed, digit-sum,
 55 SVHN/UltraMNIST sum, histogram, and CIFAR-10 histogram settings, with cautious
 56 comparison to LLP proportion-matching baselines and LLP-PVC reference results.

57 Empirically, the clearest gains appear for aggregate types where expectation or proportion matching
 58 discards information, especially digit sums and signed counts. The CIFAR-10 comparison shows
 59 that count likelihood can work well with strong pretrained features, but the fixed-instance-budget
 60 result attributes the large-bag improvement to greater sampled image exposure rather than to bag
 61 size alone.

62 2 Related Work

63 **Multiple-instance learning and count loss.** Classical multiple-instance learning (MIL) super-
 64 vises bags rather than instances Dietterich et al. [1997], Maron and Lozano-Perez [1998], Andrews
 65 et al. [2002], Carboneau et al. [2018]. The standard positive-bag assumption is a noisy OR over
 66 hidden instance labels: a bag is positive when at least one instance is positive. Neural MIL models
 67 learn bag-level predictors end-to-end Wang et al. [2018], and attention-based MIL Ilse et al. [2018]
 68 learns a bag representation through learned attention weights. These models are strong baselines for
 69 bag prediction, but attention weights are not guaranteed to be calibrated instance posteriors. Count-
 70 based extensions use the number of positive instances as a stronger bag label. Shukla et al. Shukla
 71 et al. [2023] formulate count-based weak supervision around the differentiable probability that ex-
 72 actly k Bernoulli outputs are true, and use dynamic programming to compute the resulting count
 73 loss. Related count supervision has also appeared in temporal localization Schroeter et al. [2019].
 74 Our binary Bernoulli likelihood is not new in this setting: it recovers the count probability used by
 75 Shukla et al. Binary Count-MIL is included as an equivalence check and baseline connection. Our

contribution is the finite-support convolutional generalization and its use for signed, integer-valued, posterior, and histogram aggregates.

Learning from label proportions. Learning from label proportions (LLP) observes class proportions or histograms over bags Quadrianto et al. [2009], Yu et al. [2014]. Early and applied work studied probabilistic or application-specific proportion learning Sun et al. [2017], Bortsova et al. [2018], while later deep LLP methods considered multiclass training, adversarial training, consistency regularization, pseudo-labeling, and two-stage objectives Hsu et al. [2019], Dulac-Arnold et al. [2019], Liu et al. [2019], Tsai and Lin [2020], Liu et al. [2021], Yang et al. [2021], Ma et al. [2025]. Recent theoretical LLP work studies mutual-contamination reductions, label-noise reductions, unbiased risk estimators, general losses, and sample-complexity bounds Scott and Zhang [2020], Zhang et al. [2022], Busa-Fekete et al. [2023], Wei et al. [2023], Applebaum et al. [2025], Busa-Fekete et al. [2025]. Simple proportion matching trains predicted bag proportions to match observed proportions, but it can over-smooth instance predictions. LLP-PVC Ma et al. [2026] reframes label proportions as proportional-value classification and uses efficient divide-and-conquer count-likelihood computation. Our histogram one-vs-rest count likelihood is closely related to one part of this PVC view: for each class, the class count is a Bernoulli aggregate over one-vs-rest instance probabilities. This connection also clarifies the scope of the histogram experiments. We do not present class-histogram count likelihood as an invention independent of LLP-PVC, and we do not reproduce the full LLP-PVC algorithm. Instead, the classwise count-likelihood experiments compare the shared one-vs-rest count component while the paper’s broader finite-support formulation also covers scalar integer-valued sums and signed counts.

Efficient aggregate inference. Dynamic programming computes Bernoulli count probabilities exactly but can become expensive for large bags and many classes. Recent LLP/PVC work highlights this practical issue when comparing against count-loss methods Ma et al. [2026]: exact dynamic programs are mathematically appropriate for Bernoulli counts, but their sequential structure is not the most natural fit for modern GPU training. Convolutional formulations admit parallel direct and FFT-based backends Cooley and Tukey [1965]. We implement sequential convolution for moderate-size training experiments and an FFT-tree backend for large-bag multiclass one-vs-rest count benchmarks. This makes the computational comparison a backend question: the same aggregate likelihood can be evaluated by dynamic programming, direct convolution, or FFT-tree convolution depending on support size and hardware.

3 Problem Setup

We study weakly supervised bags of instances. A training example is a bag $B = (x_1, \dots, x_n)$ together with an observed bag label y . The instance-level variables that explain y are hidden during training and are used only for evaluation. The label y may be a binary count, a signed count, an integer-valued sum, or a class histogram. The common structure is that each instance contributes a finite-support discrete value and the bag label is a sum of these contributions.

The unifying object is the *atomic support*. For a one-dimensional task, choose a finite set

$$\mathcal{A} = \{v_0, \dots, v_{S-1}\} \subset \mathbb{Z}, \quad (1)$$

where $S = |\mathcal{A}|$ is the atomic support size. A backbone model produces scores for each instance, and a contribution map turns those scores into an atomic PMF over $\mathcal{A}$. Changing $\mathcal{A}$ changes the weak-label problem while leaving the bag-prediction algorithm unchanged: $S = 2$ gives Bernoulli counts, $S = 10$ gives digit sums over $\{0, \dots, 9\}$, and $S = 3$ gives signed atoms over $\{-1, 0, +1\}$. For bag size n and contiguous support $\{0, \dots, S-1\}$, the aggregate PMF has length

$$T = n(S-1) + 1. \quad (2)$$

Signed supports use the same principle with an offset, and variable-multiplicity atoms use instance-dependent support sizes.

Let C_i denote the latent contribution of instance x_i . The contribution may also depend on observed side information a_i , such as a known sign in signed-count experiments. We assume

$$C_i \in \mathcal{S}_i \subset \mathbb{Z}^d, \quad |\mathcal{S}_i| < \infty, \quad (3)$$

Table 1: Weak-label tasks as finite-support contribution models. The atomic support size S determines the local PMF shape; convolution then gives the aggregate PMF used for the bag label.

Task	Atomic size S	Contribution support	Bag label	Aggregate width
Binary count	2	$\{0, 1\}$	number of positives	$n + 1$
Integer-valued sum	K	$\{0, \dots, K - 1\}$	scalar sum	$n(K - 1) + 1$
Signed count	3	$\{-1, 0, +1\}$	signed sum	$2n + 1$
Multiplicity	$m_i + 1$	$\{0, \dots, m_i\}$	total multiplicity	$\sum_i m_i + 1$
OVR histogram	2 per class	$\{0, 1\}$	count of class c	$n + 1$ per class

and the observed aggregate is

$$Y = \sum_{i=1}^n C_i. \quad (4)$$

For most experiments $d = 1$. For histograms, d can be the number of classes, although our scalable implementation uses one-dimensional one-vs-rest count likelihoods. The learner observes $(x_{1:n}, a_{1:n}, y)$ when side information is present, but not the individual C_i values. When discussing individual scalar tasks, we write Y_{count} for ordinary nonnegative counts, Y_{sum} for integer-valued sums such as digit sums, and $Y_{\pm}$ for signed counts. These are task-specific instances of the generic aggregate Y in Eq. (4).

Proposition 1 (Finite-support convolution law). *Let $C_1, \dots, C_n$ be conditionally independent discrete random variables given a bag $x_{1:n}$ and any observed side information $a_{1:n}$. Assume each C_i has finite support $\mathcal{S}_i \subset \mathbb{Z}^d$ and probability mass function*

$$d_i(c) = \Pr(C_i = c \mid x_i, a_i), \quad c \in \mathbb{Z}^d, \quad (5)$$

where $d_i(c) = 0$ for $c \notin \mathcal{S}_i$. Define $Y = \sum_{i=1}^n C_i$. Then Y has finite support contained in the Minkowski sum

$$\mathcal{S}_1 + \dots + \mathcal{S}_n = \{c_1 + \dots + c_n : c_i \in \mathcal{S}_i\}, \quad (6)$$

and its probability mass function is the discrete convolution

$$\Pr_{\theta}(Y = y \mid x_{1:n}, a_{1:n}) = (d_1 * \dots * d_n)(y) = \sum_{\substack{c_1 \in \mathcal{S}_1, \dots, c_n \in \mathcal{S}_n \\ c_1 + \dots + c_n = y}} \prod_{i=1}^n d_i(c_i). \quad (7)$$

Proposition 1 is a standard probability identity, but making it explicit clarifies the modeling choice. Once the atomic contribution distributions d_i are predicted by a neural network, the bag-label distribution is determined exactly by convolution. Different weak-label tasks differ mainly in the support $\mathcal{S}_i$ and in which side information is observed.

Binary Count-MIL is therefore the simplest case of Eq. (4); it is not new by itself. The broader formulation is useful because signed counts, integer-valued sums, multiplicity counts, and one-vs-rest histograms can be handled by changing the finite-support atom while keeping the same convolutional bag-prediction principle. In our experiments, bags are sampled on the fly with replacement from the corresponding train or test split. Thus an image may appear in multiple bags, but train and test bags are always drawn from disjoint dataset splits.

4 Convolutional Bag Prediction

4.1 Aggregate Likelihood

Following the setup in Section 3, the instance network emits the atomic PMFs d_i . For one-dimensional contributions, this is a categorical distribution over a finite integer support:

$$d_i(k) = p_{\theta}(z_i = k \mid x_i), \quad k \in \{a, \dots, b\}. \quad (8)$$

By Proposition 1, the exact bag-label likelihood is the convolution of these atomic PMFs:

$$p_{\theta}(Y = y \mid x_{1:n}) = (d_1 * d_2 * \dots * d_n)(y). \quad (9)$$

151 Training minimizes the negative log likelihood

$$\mathcal{L}_{\text{agg}}(\theta) = -\log p_{\theta}(Y = y \mid x_{1:n}). \quad (10)$$

152 The support of the aggregate PMF is finite and contiguous:

$$\text{supp}(Y) = \{na, na + 1, \dots, nb\}. \quad (11)$$

153 Equivalently, if each atom has contiguous support size $S = b - a + 1$, then the aggregate PMF
 154 has width $T = n(S - 1) + 1$ after shifting by na . All experiments use exact likelihoods over this
 155 support; approximations enter only through the neural instance model, not through sampling latent
 156 assignments.

157 **Two immediate consequences.** First, when the atomic support is Bernoulli, the convolutional
 158 likelihood is exactly the coefficient of the polynomial $\prod_i ((1 - p_i) + p_i t)$ and therefore recovers
 159 the standard count probability. Second, for any finite support, the likelihood uses the entire induced
 160 aggregate distribution, not only the mean aggregate value. Thus two instance predictors with the
 161 same expected sum can receive different likelihoods if they assign different probability mass to the
 162 observed aggregate. This is the distinction exploited by the digit-sum and signed-count experiments.

163 4.2 Training Objective

164 For each minibatch, the instance model first produces atomic PMFs for every instance in every bag.
 165 The aggregate layer then deterministically maps these atoms to an exact PMF over the observed bag
 166 label. This separation makes the aggregate objective backbone-agnostic: an MLP, CNN, transformer,
 167 or pretrained image encoder can be used as long as it produces per-instance scores from which Φ
 168 can form the required finite-support PMFs. The training loss is the negative log probability assigned
 169 to the observed aggregate,

$$\mathcal{L}(\theta) = -\frac{1}{B} \sum_{b=1}^B \log(d_{b1} * \dots * d_{bn_b})(y_b). \quad (12)$$

170 Gradients are backpropagated through the convolutional aggregate layer into the instance network.
 171 Instance labels are never used for training in the weak-supervision experiments; they are used only
 172 for evaluation of hidden instance recovery. This is different from expected-value regression base-
 173 lines, which regress the scalar expectation $\sum_i \mathbb{E}[z_i]$ to the aggregate label and therefore discard the
 174 higher-order distributional information in the count or sum.

175 4.3 Special Cases

176 **Binary counts.** For a binary target concept c^* , define the hidden instance label $z_i = \mathbf{1}\{a_i = c^*\}$,
 177 where a_i is the underlying class of instance i . In the binary MNIST experiments, c^* is a chosen digit,
 178 so images of that digit are positive and all other MNIST digits are negative; training observes only
 179 the bag count, not the individual z_i . For Bernoulli target indicators,

$$d_i = (1 - p_i, p_i), \quad Y_{\text{count}} = \sum_i z_i. \quad (13)$$

180 Equivalently, each instance contributes a length-two one-dimensional kernel over support $\{0, 1\}$:

$$\kappa_i = [1 - p_i, p_i]. \quad (14)$$

181 The probability of observing count y is the coefficient of t^y in

$$\prod_{i=1}^n ((1 - p_i) + p_i t). \quad (15)$$

182 This recovers the standard binary count likelihood.

Algorithm 1 Convolutional bag prediction for one minibatch

Require: Bags $\{(x_{b1:n_b}, a_{b1:n_b}, y_b)\}_{b=1}^B$, backbone f_θ , contribution map Φ , backend Conv

Ensure: Loss $\mathcal{L}$ and bag-label probabilities $\{p_b\}_{b=1}^B$

```

1: for  $b = 1, \dots, B$  do
2:    $H_b \leftarrow f_\theta(x_{b1:n_b})$  ▷ instance logits or class scores
3:   for  $i = 1, \dots, n_b$  do
4:      $d_{bi} \leftarrow \Phi(H_{bi}, a_{bi})$  ▷ atomic PMF on the task support
5:   end for
6:   if histogram one-vs-rest objective then
7:     for  $c = 1, \dots, C$  do
8:        $m_{bc} \leftarrow \text{Conv}(\{[1 - p_{bic}, p_{bic}]\}_{i=1}^{n_b})$ 
9:        $p_{bc} \leftarrow m_{bc}(h_{bc})$ 
10:    end for
11:     $p_b \leftarrow \prod_{c=1}^C p_{bc}^{1/C}$ 
12:     $\ell_b \leftarrow -C^{-1} \sum_{c=1}^C \log p_{bc}$ 
13:  else
14:     $m_b \leftarrow \text{Conv}(\{d_{bi}\}_{i=1}^{n_b})$  ▷ sequential, grouped, or FFT-tree
15:     $p_b \leftarrow m_b(y_b)$ 
16:     $\ell_b \leftarrow -\log p_b$ 
17:  end if
18: end for
19: return  $\mathcal{L} = B^{-1} \sum_{b=1}^B \ell_b, \{p_b\}_{b=1}^B$ 

```

183 **Signed counts.** For known signs $s_i \in \{-1, +1\}$ and latent target indicators $z_i \in \{0, 1\}$,

$$Y_\pm = \sum_i s_i z_i. \quad (16)$$

184 Signed MNIST uses the same target-digit construction as binary MNIST, but each instance also has
 185 an observed sign. Non-target digits contribute 0, while a target-digit image contributes either +1 or
 186 −1 according to its sign. Each atom has support $\{-1, 0, +1\}$ after incorporating the sign:

$$d_i = \begin{cases} (1 - p_i)\delta_0 + p_i\delta_{+1}, & s_i = +1, \\ (1 - p_i)\delta_0 + p_i\delta_{-1}, & s_i = -1. \end{cases} \quad (17)$$

187 In array order $[-1, 0, +1]$, the corresponding signed kernels are

$$\kappa_i^{(+)} = [0, 1 - p_i, p_i], \quad \kappa_i^{(-)} = [p_i, 1 - p_i, 0]. \quad (18)$$

188 This setting exposes cancellation: many distinct latent assignments can yield the same observed
 189 aggregate, especially when $Y_\pm = 0$.

190 **Integer-valued sums.** For hidden finite integer values $z_i \in \{0, \dots, K-1\}$, the observed bag label
 191 is

$$Y_{\text{sum}} = \sum_i z_i, \quad (19)$$

192 so the label is a scalar sum rather than a class proportion. Unlike expected-sum regression, the like-
 193 lihood uses the full induced distribution over possible sums. If a model predicts digit probabilities
 194 $p_\theta(z_i = k \mid x_i)$, the likelihood compares the observed sum against the full convolved PMF rather
 195 than only the mean $\sum_i \mathbb{E}[z_i]$. The one-dimensional kernel for instance i is the full class-probability
 196 vector

$$\kappa_i = [p_{i0}, p_{i1}, \dots, p_{i,K-1}], \quad (20)$$

197 so the aggregate support has width $n(K-1) + 1$. For MNIST digit-sum, $K = 10$, z_i is the original
 198 digit value in $\{0, \dots, 9\}$, and the observed bag label is the sum of those hidden digit values.

199 **Histogram labels and one-vs-rest counts.** For multiclass histograms, each class count can be
 200 treated as a one-vs-rest Bernoulli count. Let p_{ic} be the predicted probability of class c for instance i ,
 201 and let h_c be the observed count of class c in the bag. This differs from ordinary proportion matching.

CE, KL, or MSE baselines first average instance probabilities into a predicted bag-proportion vector

$$\bar{p}_c = \frac{1}{n} \sum_{i=1}^n p_{ic}, \quad \hat{q} = (\bar{p}_1, \dots, \bar{p}_C), \quad (21)$$

and compare $\hat{q}$ to the observed proportion vector $q = (h_1/n, \dots, h_C/n)$. For example, KL proportion matching minimizes $\sum_c q_c \log(q_c/\bar{p}_c)$, while MSE minimizes $\sum_c (q_c - \bar{p}_c)^2$. These losses match only the expected class proportions. For every class c , we form Bernoulli count kernels

$$\kappa_{ic} = [1 - p_{ic}, p_{ic}] \quad (22)$$

and compute a classwise count likelihood for the observed integer count h_c . The one-vs-rest count objective is

$$\mathcal{L}_{\text{hist}}(\theta) = -\frac{1}{C} \sum_{c=1}^C \log \left[\prod_{i=1}^n ((1 - p_{ic}) + p_{ic} t) \right]_{t^{h_c}}, \quad (23)$$

where $[\cdot]_{t^{h_c}}$ denotes coefficient extraction. This objective is closely related to the count-likelihood component of LLP-PVC, while sharing the same finite-support convolution backend used elsewhere in this paper. In the experiments, the one-vs-rest rows denote this classwise count likelihood rather than the full LLP-PVC algorithm. Unlike a full joint multiclass histogram likelihood, this one-vs-rest objective does not enforce the dependencies among class counts exactly. It is nevertheless a scalable comparison point and exposes the same one-dimensional count backend used by the other aggregate likelihoods.

4.4 Posterior Marginals

For binary counts, we compute count-conditioned posterior marginals

$$q_i = p_\theta(z_i = 1 \mid \sum_j z_j = y, x_{1:n}), \quad (24)$$

using prefix/suffix count distributions. Let $F_{i-1}(k)$ be the probability that the first $i-1$ instances contain k positives, and let $B_{i+1}(k)$ be the probability that instances $i+1, \dots, n$ contain k positives. Then

$$q_i = \frac{p_i \sum_k F_{i-1}(k) B_{i+1}(y-1-k)}{\sum_k F_{i-1}(k) ((1-p_i) B_{i+1}(y-k) + p_i B_{i+1}(y-1-k))}. \quad (25)$$

These marginals are used as additional information extracted from the count label after likelihood training. They support instance recovery diagnostics and uncertainty analysis, but they are not part of the core training objective in this paper.

4.5 Computational Backends

All of the above objectives are one-dimensional convolutions over integer aggregate support. If $m^{(i)}$ denotes the aggregate PMF after processing the first i atoms, then

$$m^{(i)}(s) = \sum_{r=a}^b m^{(i-1)}(s-r) d_i(r), \quad m^{(0)} = \delta_0. \quad (26)$$

This recurrence is the same probability law whether implemented as dynamic programming, shifted dense tensor accumulation, grouped `conv1d`, or FFT polynomial multiplication. The choice of backend is therefore computational rather than statistical: the likelihood being optimized is unchanged, but the hardware profile differs. The implementation uses several exact backends:

- Sequential finite-support convolution accumulates shifted copies of the current PMF and is used for binary, signed, and integer-valued sum experiments.
- Dynamic programming is used as a Shukla-style Bernoulli count baseline; it computes the same binary count coefficients as the convolutional view.
- Grouped GPU convolution packs signed Bernoulli groups into channels and applies grouped `conv1d`.
- Balanced FFT-tree convolution multiplies the atomic polynomials pairwise and is used for runtime experiments and multiclass one-vs-rest histogram likelihoods.

Table 2: Exact aggregate backends. n is bag size, S is atomic support size, $T = n(S - 1) + 1$ is aggregate PMF width, and C is number of one-vs-rest classes.

Backend	Setting	Work per bag	Use
DP	Bernoulli count	$O(nT)$	Shukla-style count baseline
Sequential convolution	finite support	$O(nST)$	main moderate-bag experiments
Grouped convolution	signed Bernoulli	$O(nT)$, groups	batched signed GPU implementation
FFT-tree	finite support / OVR	$O(CT \log T \log n)$	runtime, OVR histograms

Grouped signed conv1d. The literal GPU conv1d backend is used for grouped signed binary aggregates. Each active group starts as a point mass at zero on support $[-K_{\max}, K_{\max}]$. For one signed instance, the backend applies a length-three channelwise kernel and padding one. Because PyTorch’s conv1d operator is cross-correlation rather than mathematical convolution, the code stores the kernel in the order

$$[p_i \mathbf{1}\{s_i = +1\}, 1 - p_i, p_i \mathbf{1}\{s_i = -1\}], \quad (27)$$

so the left kernel entry shifts probability mass toward larger aggregate values and the right entry shifts mass toward smaller values. This is only an implementation orientation; in mathematical support order $[-1, 0, +1]$ the signed kernels are the atoms shown above.

Multiclass and histogram implementation. For digit-sum, the atom width is the number of finite digit values, for example ten for MNIST digits. For histogram experiments, we avoid a full C -dimensional joint histogram over classes and instead compute C one-vs-rest one-dimensional count likelihoods in parallel. This keeps the aggregate support one-dimensional for each class and makes large bags practical on GPU. The FFT-tree backend is useful for this setting because the same pairwise polynomial products are batched over classes and bags. Padding or masked positions are represented by deterministic zero-contribution atoms, such as $[1, 0]$ for Bernoulli counts, $[0, 1, 0]$ for signed support $[-1, 0, +1]$, and $[1, 0, \dots, 0]$ for integer-valued sums.

5 Experiments

5.1 Experimental Setup

We evaluate aggregate-supervised instance learning under five aggregate-label themes:

1. binary count labels;
2. integer-valued sum labels;
3. signed count labels;
4. histogram or class-count labels;
5. runtime of exact aggregate backends.

Unless otherwise stated, MNIST-family experiments use LeNet-style instance networks, bags are sampled on the fly, and tables report means over three random seeds. Hidden instance labels are never used for training; they are used only for evaluation. Implementation details are summarized in Appendix B. Within each theme, additional datasets such as FashionMNIST, SVHN, UltraMNIST, or CIFAR-10 are used as supporting evidence rather than separate experiment families. Each subsection changes the observed aggregate label while keeping the same finite-support convolutional likelihood interface.

Metrics. Across the experimental tables, MAE columns report the mean absolute difference between the predicted aggregate and the observed aggregate over test bags. When the model outputs a full aggregate PMF, we report either the maximum-probability aggregate or the PMF expectation as indicated by the table label. For histograms, count MAE is the mean absolute class-count error

Table 3: Binary count labels. Count likelihood recovers accurate hidden target detectors on MNIST target-count bags.

Data	Bag/train	Method	Count MAE	Hidden acc.
MNIST	10/1000	exact count	0.115	0.988
MNIST	10/5000	exact count	0.090	0.991
MNIST	50/1000	exact count	0.686	0.987
MNIST	50/5000	exact count	0.460	0.991

Table 4: Integer-valued sum labels. Exact likelihood uses the full convolved PMF over sums, whereas MSE regresses only the expected sum. SVHN rows report two-seed means; UltraMNIST reports best-checkpoint mean $\pm$ std. over five seeds.

Data	Bag/train	Method	Sum MAE	Hidden acc.
MNIST	10/1000	exact sum	0.612	.982
MNIST	10/1000	MSE sum	2.535	.226
MNIST	10/5000	exact sum	0.277	.992
MNIST	10/5000	MSE sum	1.023	.746
MNIST	50/1000	exact sum	4.901	.693
MNIST	50/1000	MSE sum	6.274	.248
MNIST	50/5000	exact sum	2.229	.887
MNIST	50/5000	MSE sum	2.594	.817
SVHN	10/5000	exact sum, scratch	5.812	.229
SVHN	10/5000	exact sum, pretr.	1.467	.952
UltraMNIST	3-5/800	exact sum	.341 $\pm$.038	.974 $\pm$.003

274 averaged over classes and bags,

$$\frac{1}{N} \sum_B \frac{1}{C} \sum_{c=1}^C |\hat{h}_{Bc} - h_{Bc}|, \quad (28)$$

275 Rows labelled MSE denote MSE training objectives; MAE and accuracy columns are evaluation
 276 metrics computed after training. Hidden-instance AUC or accuracy is never used for model selection
 277 unless explicitly stated.

278 5.2 Binary Count Labels

279 Plain count-likelihood training recovers high hidden-instance accuracy across bag sizes. For MNIST-
 280 Bags, we make MNIST binary by choosing digit 9 as the positive class and treating digits 0-8 as
 281 negative. The learner observes the number of target-digit images in the bag; hidden instance labels
 282 are used only for evaluation. This validation case verifies that the convolutional formulation behaves
 283 like the established Bernoulli count probability while sharing the same implementation pattern as
 284 the new aggregate types. FashionMNIST target-count results are recorded in the experiment report;
 285 the main table uses MNIST so that the hidden-instance metric is consistent with the other result
 286 tables.

287 5.3 Integer-Valued Sum Labels

288 Digit-sum supervision is an instance of integer-valued aggregate labeling: the learner observes a sin-
 289 gle scalar while each hidden instance label has ten possible values. Here z_i is the original MNIST
 290 digit value in $\{0, \dots, 9\}$ and the bag label is $Y_{\text{sum}} = \sum_i z_i$. Results show that exact finite-support
 291 likelihood recovers hidden digit predictors more efficiently than matching only the expected aggre-
 292 gate. For comparison, we report a simple MSE baseline that regresses the scalar expected sum
 293 $\sum_i \mathbb{E}[z_i]$.

294 For $n = 10$ and 1000 training bags, the MSE baseline has substantially worse aggregate error and
 295 hidden digit recovery than the exact likelihood. With more training bags it improves, but the exact
 296 likelihood remains stronger in this table. For $n = 50$, the MSE baseline can recover some hidden
 297 digit structure while still making large aggregate errors, because small expectation errors accumulate

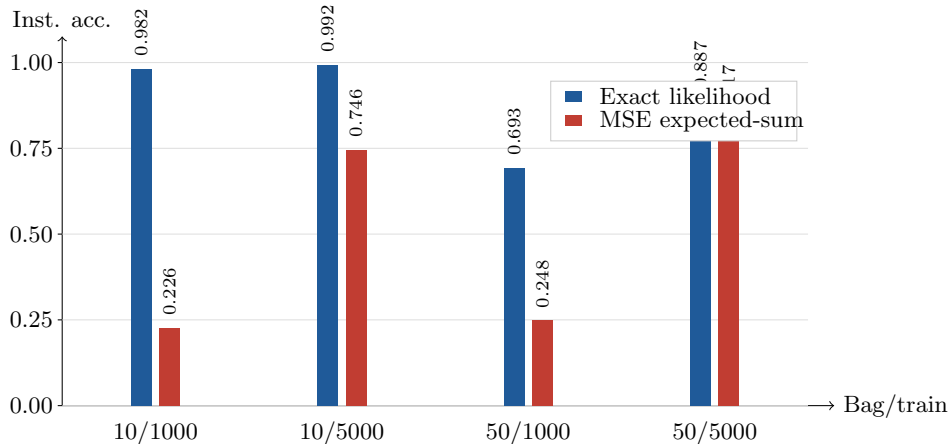


Figure 2: Hidden digit recovery from digit-sum labels. Exact finite-support likelihood uses the full convolved sum distribution, while MSE matches only the expected sum. Instance labels are used only for evaluation.

Table 5: Signed count labels with random known signs. Signed sums are many-to-one because positive and negative contributions can cancel, but the likelihood still recovers accurate hidden target detectors.

Data	Bag/train	Method	Signed MAE	Hidden acc.
MNIST	10/1000	exact signed	0.095	0.990
MNIST	10/5000	exact signed	0.045	0.996
MNIST	50/1000	exact signed	0.243	0.994
MNIST	50/5000	exact signed	0.198	0.996

across many digits. SVHN and UltraMNIST strengthen the same digit-sum story beyond MNIST: pretraining makes natural RGB house-number digit sums usable, while UltraMNIST gives a clean-patch benchmark with high hidden digit recovery. These rows support the interpretation of Eq. (2): changing the finite support and image domain creates a new sum-label problem without changing the aggregate likelihood layer.

5.4 Signed Count Labels

Signed counts test an aggregate that is not reducible to ordinary class proportions. We again use the digit-9 target indicator, but each instance carries a known random sign; target-digit images contribute $+1$ or -1 , while all other digits contribute 0. Even with random signs, the observed signed sum is many-to-one: positive and negative contributions can cancel, so different latent assignments can yield the same bag label. The likelihood nevertheless yields accurate hidden target detectors, especially with more training bags. FashionMNIST signed-count results are recorded in the experiment report; the main table uses MNIST so that the hidden-instance metric is consistent with the other result tables.

5.5 Histogram Count Labels

Histogram supervision is richer than scalar sums, and LLP baselines are strong on MNIST-family data. For MNIST histograms, the hidden category is the original digit class and the observed label is the vector of digit counts in the bag. These experiments compare exact one-vs-rest count likelihood (OVR count) with standard LLP-style proportion-matching objectives across MNIST and CIFAR-10 histogram labels. Table 6 reports the histogram results. The main table focuses on MSE proportion matching as a simple expected-histogram baseline and compares it with the exact one-vs-rest count likelihood. The advantage is clearest for larger bags: at $n = 50$ and 1000 training bags, one-vs-rest count likelihood improves digit accuracy from 11.6% to 98.5% and reduces count MAE

Table 6: Histogram supervision across datasets. Count MAE is mean absolute class-count error. OVR count denotes the classwise one-vs-rest count likelihood in Section 4; MSE is a proportion-matching LLP objective. CIFAR-10 pretrained rows use ImageNet initialization.

Data	Bag/train	Backbone	Method	Count MAE	Hidden acc.
MNIST	10/1000	LeNet	MSE	.163	.962
MNIST	10/1000	LeNet	OVR count	.032	.983
MNIST	10/5000	LeNet	MSE	.127	.974
MNIST	10/5000	LeNet	OVR count	.016	.992
MNIST	50/1000	LeNet	MSE	1.683	.116
MNIST	50/1000	LeNet	OVR count	.137	.985
MNIST	50/5000	LeNet	MSE	1.684	.116
MNIST	50/5000	LeNet	OVR count	.080	.992
CIFAR-10	16/1000	scratch R18	MSE	.825	.476
CIFAR-10	16/1000	scratch R18	OVR count	.838	.527
CIFAR-10	64/1000	scratch R18	MSE	2.013	.270
CIFAR-10	64/1000	scratch R18	OVR count	1.855	.496
CIFAR-10	128/1000	scratch R18	MSE	2.829	.226
CIFAR-10	128/1000	scratch R18	OVR count	2.707	.488
CIFAR-10	16/1000	pretrained R18	OVR count	.260	.902
CIFAR-10	64/1000	pretrained R18	OVR count	.678	.914
CIFAR-10	64/250	pretrained R18	OVR count	1.115	.803
CIFAR-10	64/250	pretrained R18	best hist.	1.013	.806

from 1.683 to 0.137. Additional CE/KL LLP baselines are strong on MNIST histograms and are recorded in the experiment report; we omit them here to keep the main table focused on the effect of replacing squared-error aggregate matching with an exact count likelihood. CIFAR-10 with ResNet-18 evaluates the same histogram objectives in a standard image benchmark. Prior LLP-PVC results use a ResNet-18 backbone, and Ma et al.’s appendix specifies ImageNet-pretrained initialization. We evaluate both randomly initialized and ImageNet-pretrained ResNet-18 backbones because the representation source has a large effect on LLP performance. The from-scratch setting evaluates the same count-likelihood pipeline without external pretrained features, while the pretrained setting matches the backbone initialization specified for LLP-PVC.

The from-scratch results are substantially below the pretrained setting, confirming that backbone initialization is a major factor in this comparison. With ImageNet-pretrained ResNet-18 and the one-vs-rest count objective, the $n = 16$ and $n = 64$ settings reach about 90.2% and 91.4% final instance accuracy, respectively. However, the $n = 64$, train 1000 setting exposes four times as many image instances per epoch as $n = 16$, train 1000. The fixed-instance-budget setting, $n = 64$ with 250 train bags, reaches only about 80.3% final instance accuracy and 80.6% when selecting by observed histogram MAE. Thus the CIFAR-10 results show that count likelihood benefits from a strong pretrained representation. They also show that more aggregate-labeled bags or sampled image instances can improve hidden-category recovery under this protocol, while the $n = 64$ improvement is explained by greater total image exposure or aggregate observations rather than bag size alone.

The CIFAR-100 coarse experiment used 20 coarse classes, bag size 50, train bags 1000, and two seeds. One-vs-rest count likelihood reaches about 30% hidden coarse-class accuracy, above the 5% random baseline. CIFAR-10 provides the direct LLP-PVC comparison, while CIFAR-100 coarse labels probe a higher-class-count setting.

5.6 Runtime

Our FFT-tree backend computes batched multiclass one-vs-rest count PMFs on GPU and gives large practical speedups over CPU dynamic programming in large-bag benchmarks. This addresses a practical limitation of DP-based count losses: exact count probabilities need not be tied to a sequential CPU implementation. The benchmark isolates aggregate-PMF evaluation; it measures backend speed rather than end-to-end training time or representation quality.

6 Discussion

The experiments support a hierarchy of aggregate supervision. The clearest gains occur when the observed aggregate is not well represented by only an expectation or a class proportion. Scalar expected-value matching is weak in low-data digit-sum settings, whereas exact aggregate likelihood

Table 7: Runtime for multiclass one-vs-rest count PMFs with batch size 256 and $C = 10$ classes. CPU DP is compared to GPU FFT-tree convolution.

Bag size	CPU DP (s)	GPU FFT-tree (s)	Max abs. error
64	8.246	0.00097	3.4×10^{-7}
128	17.230	0.00097	4.2×10^{-7}
256	36.785	0.00121	5.1×10^{-7}
512	54.040	0.00134	6.6×10^{-7}

can recover accurate instance predictors from the same scalar labels. With more training bags and small bags, expected-value baselines can become competitive, so the observed advantage is data efficiency and likelihood fidelity rather than universal dominance. The SVHN and UltraMNIST rows strengthen the digit-sum evidence: the same integer-valued sum likelihood remains useful when the support is fixed at $S = 10$ but the image domain and backbone change. Together with binary and signed benchmarks, these results support the view that FS-Conv is a generic finite-support sum-label layer rather than a collection of task-specific losses.

Histogram labels are substantially richer, and standard LLP/proportion-matching baselines can already be strong. On MNIST histogram supervision, exact one-vs-rest count likelihood improves over CE/KL proportion matching, especially for larger bags, suggesting that likelihood fidelity can still matter when the aggregate label is rich. This distinction defines the scope of the method: it provides a unified likelihood framework for finite-support aggregate labels beyond ordinary binary counts and class proportions, while standard LLP objectives remain strong when the observed label is already a rich class histogram.

The same formulation is modular with respect to the instance model. Backbone choice determines the quality of the atomic PMFs, while the aggregate likelihood is determined by the weak label: binary counts, signed counts, integer-valued sums, and one-vs-rest histograms all use the same convolutional bag-prediction interface. This modularity is useful computationally as well as conceptually, because the exact aggregate PMF can be evaluated by CPU DP, direct convolution, grouped convolution, or GPU FFT-tree multiplication without changing the statistical objective.

Several limitations remain. Large bags can expose calibration issues even when instance ranking is accurate. Signed aggregates can be ambiguous because different latent assignments may yield the same observed sum. CIFAR-scale experiments require matching the backbone initialization and training protocol used by recent LLP work. Pretrained ResNet-18 one-vs-rest count likelihood is effective for $n = 16$ and $n = 64$, but the fixed-instance-budget comparison shows that the large-bag gain disappears when total sampled image exposure is matched. These results show effective use of aggregate-labeled image volume under a strong representation, but they do not indicate intrinsic large-bag superiority or superiority to fully supervised learning at matched annotation budget.

References

- Stuart Andrews, Ioannis Tsochantaridis, and Thomas Hofmann. Support vector machines for multiple-instance learning. In *Advances in Neural Information Processing Systems*, 2002.
- Lorne Applebaum, Travis Dick, Claudio Gentile, Haim Kaplan, and Tomer Koren. Optimal learning from label proportions with general loss functions, 2025.
- Gerda Bortsova, Florian Dubost, Silas Orting, Ioannis Katramados, Laurens Hogeweg, Laura Thomsen, Mathilde Wille, and Marleen de Bruijne. Deep learning from label proportions for emphysema quantification. *Medical Image Computing and Computer Assisted Intervention*, pages 768–776, 2018.
- Robert Busa-Fekete, Heejin Choi, Travis Dick, Claudio Gentile, and Andres Munoz Medina. Easy learning from label proportions. In *Advances in Neural Information Processing Systems*, pages 14957–14968, 2023.
- Robert Busa-Fekete, Travis Dick, Claudio Gentile, Haim Kaplan, Tomer Koren, and Uri Stemmer. Nearly optimal sample complexity for learning with label proportions, 2025.

- 396 Marc-Andre Carbonneau, Veronika Cheplygina, Eric Granger, and Ghyslain Gagnon. Multiple in-
397 stance learning: A survey of problem characteristics and applications. *Pattern Recognition*, 77:
398 329–353, 2018.
- 399 James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex fourier
400 series. *Mathematics of Computation*, 19(90):297–301, 1965.
- 401 Thomas G. Dietterich, Richard H. Lathrop, and Tomas Lozano-Perez. Solving the multiple instance
402 problem with axis-parallel rectangles. *Artificial Intelligence*, 89(1–2):31–71, 1997.
- 403 Gabriel Dulac-Arnold, Neil Zeghidour, Marco Cuturi, Lucas Beyer, and Jean-Philippe Vert. Deep
404 multi-class learning from label proportions, 2019.
- 405 Yen-Chang Hsu, Zhaoyang Lv, Joel Schlosser, Phillip Odom, and Zsolt Kira. Multi-class classifica-
406 tion without multi-class labels. In *International Conference on Learning Representations*, 2019.
- 407 Maximilian Ilse, Jakub M. Tomczak, and Max Welling. Attention-based deep multiple instance
408 learning. In *International Conference on Machine Learning*, 2018.
- 409 Jiabin Liu, Bo Wang, Zhiquan Qi, Yingjie Tian, and Yong Shi. Learning from label proportions with
410 generative adversarial networks. In *Advances in Neural Information Processing Systems*, 2019.
- 411 Jiabin Liu, Bo Wang, Xin Shen, Zhiquan Qi, and Yingjie Tian. Two-stage training for learning from
412 label proportions. In *International Joint Conference on Artificial Intelligence*, pages 2737–2743,
413 2021.
- 414 Tianhao Ma, Han Chen, Juncheng Hu, Yungang Zhu, and Ximing Li. Forming auxiliary high-
415 confident instance-level loss to promote learning from label proportions. In *IEEE/CVF Confer-*
416 *ence on Computer Vision and Pattern Recognition*, pages 20592–20601, 2025.
- 417 Tianhao Ma, Wei Wang, Ximing Li, Gang Niu, and Masashi Sugiyama. Learning from label propor-
418 tions via proportional value classification. In *International Conference on Learning Representa-*
419 *tions*, 2026.
- 420 Oded Maron and Tomas Lozano-Perez. A framework for multiple-instance learning. In *Advances*
421 *in Neural Information Processing Systems*, 1998.
- 422 Novi Quadrianto, Alex J. Smola, Tiberio S. Caetano, and Quoc V. Le. Estimating labels from label
423 proportions. *Journal of Machine Learning Research*, 10:2349–2374, 2009.
- 424 Julian Schroeter, Kirill A. Sidorov, and David Marshall. Weakly-supervised temporal localization
425 via occurrence count learning. In *International Conference on Machine Learning*, pages 5649–
426 5659, 2019.
- 427 Clayton Scott and Jianxin Zhang. Learning from label proportions: A mutual contamination frame-
428 work. In *Advances in Neural Information Processing Systems*, volume 33, pages 22256–22267,
429 2020.
- 430 Vinay Shukla, Zhe Zeng, Kareem Ahmed, and Guy Van den Broeck. A uni-
431 fied approach to count-based weakly supervised learning. In *Advances in Neu-*
432 *ral Information Processing Systems*, volume 36, pages 38709–38722, 2023.
433 URL [https://proceedings.neurips.cc/paper_files/paper/2023/hash/](https://proceedings.neurips.cc/paper_files/paper/2023/hash/79a0c8e7ae8e403e39341ea6b0ba4c21-Abstract-Conference.html)
434 [79a0c8e7ae8e403e39341ea6b0ba4c21-Abstract-Conference.html](https://proceedings.neurips.cc/paper_files/paper/2023/hash/79a0c8e7ae8e403e39341ea6b0ba4c21-Abstract-Conference.html).
- 435 Tao Sun, Dan Sheldon, and Brendan O’Connor. A probabilistic approach for learning with label
436 proportions applied to the us presidential election. In *IEEE International Conference on Data*
437 *Mining*, pages 445–454, 2017.
- 438 Kuen-Han Tsai and Hsuan-Tien Lin. Learning from label proportions with consistency regulariza-
439 tion. In *Asian Conference on Machine Learning*, pages 513–528, 2020.
- 440 Xinggang Wang, Yongluan Yan, Peng Tang, and Wenyu Liu. Revisiting multiple instance neural
441 networks. *Pattern Recognition*, 74:15–24, 2018.

- 442 Zixi Wei, Lei Feng, Bo Han, Tongliang Liu, Gang Niu, Xiaofeng Zhu, and Heng Tao Shen. A univer-
 443 sal unbiased method for classification from aggregate observations. In *International Conference*
 444 *on Machine Learning*, pages 36804–36820, 2023.
- 445 Haoran Yang, Wanjing Zhang, and Wai Lam. A two-stage training framework with feature-label
 446 matching mechanism for learning from label proportions. In *Asian Conference on Machine Learn-*
 447 *ing*, pages 1461–1476, 2021.
- 448 Felix X. Yu, Krzysztof Choromanski, Sanjiv Kumar, Tony Jebara, and Shih-Fu Chang. On learning
 449 from label proportions, 2014.
- 450 Jianxin Zhang, Yutong Wang, and Clayton Scott. Learning from label proportions by learning with
 451 label noise. In *Advances in Neural Information Processing Systems*, pages 26933–26942, 2022.

452 A Additional Derivations

453 For binary count labels, let $p_i = p_\theta(z_i = 1 \mid x_i)$ and let the observed count be y . Define the prefix
 454 distribution $F_i(k) = p_\theta(\sum_{j=1}^i z_j = k \mid x_{1:i})$ and suffix distribution $B_i(k) = p_\theta(\sum_{j=i}^n z_j = k \mid$
 455 $x_{i:n})$. They are initialized by $F_0(0) = 1$ and $B_{n+1}(0) = 1$ and updated by the Bernoulli count
 456 recurrence

$$F_i(k) = (1 - p_i)F_{i-1}(k) + p_iF_{i-1}(k - 1), \quad (29)$$

$$B_i(k) = (1 - p_i)B_{i+1}(k) + p_iB_{i+1}(k - 1). \quad (30)$$

457 Conditioning on the observed count gives

$$p_\theta(z_i = 1 \mid \sum_j z_j = y, x_{1:n}) = \frac{p_i \sum_k F_{i-1}(k) B_{i+1}(y - 1 - k)}{\sum_k F_{i-1}(k) [(1 - p_i) B_{i+1}(y - k) + p_i B_{i+1}(y - 1 - k)]}. \quad (31)$$

458 The denominator is the bag likelihood $p_\theta(Y = y \mid x_{1:n})$ written with instance i separated out. In the
 459 unsigned binary case, the resulting posterior marginals sum to the observed count up to numerical
 460 precision.

461 B Implementation Details

462 All weak-supervision experiments sample bags on the fly rather than pregenerating fixed bag files.
 463 Instance labels are used only for evaluation metrics such as hidden-instance AUC or hidden digit
 464 accuracy. MNIST-family experiments use LeNet-style instance classifiers, Adam with learning rate
 465 5×10^{-4} , weight decay 10^{-4} , and 50 epochs. The main MNIST-family configurations use seeds
 466 0, 1, 2, test on 1000 sampled bags, and evaluate mean bag sizes 10 and 50 with standard deviations
 467 2 and 10.

468 Histogram count-likelihood experiments treat each class count as a one-vs-rest Bernoulli count like-
 469 lihood. The MNIST histogram experiments use batch size 192 for large-bag configurations. CIFAR-
 470 10 experiments use ResNet-18 backbones. The from-scratch CIFAR settings use randomly initial-
 471 ized ResNet-18. The pretrained one-vs-rest count settings use ImageNet-pretrained ResNet-18 with
 472 random crop and horizontal flip augmentation, SGD with momentum 0.9, weight decay 10^{-4} , learn-
 473 ing rate 5×10^{-4} , and a cosine learning-rate schedule for 500 epochs. The pretrained CIFAR-10
 474 analysis is limited to the evaluated $n = 16$ and $n = 64$ one-vs-rest count settings.

475 C Additional Tables

476 We also evaluated posterior-training variants for binary MNIST-Bags, including hard and tempered
 477 posterior objectives. These variants did not reliably improve the negative-log-likelihood result, so
 478 the main method treats posterior marginals as an inference and diagnostic tool rather than as the
 479 central training objective. The main text reports aggregate means, and seed counts are shown for
 480 comparisons evaluated with fewer than three seeds.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”,**
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers.**

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer **[N/A]** means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer **[N/A]** means that the paper does not include experiments.
- The authors should answer **[Yes]** if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer **[N/A]** means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- 687 • The paper should disclose whether the full research project required more compute
688 than the experiments reported in the paper (e.g., preliminary or failed experiments
689 that didn't make it into the paper).

690 **9. Code of ethics**

691 Question: Does the research conducted in the paper conform, in every respect, with the
692 NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

693 Answer: **[TODO]**

694 Justification: **[TODO]**

695 Guidelines:

- 696 • The answer [N/A] means that the authors have not reviewed the NeurIPS Code of
697 Ethics.
698 • If the authors answer [No], they should explain the special circumstances that require
699 a deviation from the Code of Ethics.
700 • The authors should make sure to preserve anonymity (e.g., if there is a special consid-
701 eration due to laws or regulations in their jurisdiction).

702 **10. Broader impacts**

703 Question: Does the paper discuss both potential positive societal impacts and negative
704 societal impacts of the work performed?

705 Answer: **[TODO]**

706 Justification: **[TODO]**

707 Guidelines:

- 708 • The answer [N/A] means that there is no societal impact of the work performed.
709 • If the authors answer [N/A] or [No], they should explain why their work has no soci-
710 etal impact or why the paper does not address societal impact.
711 • Examples of negative societal impacts include potential malicious or unintended uses
712 (e.g., disinformation, generating fake profiles, surveillance), fairness considerations
713 (e.g., deployment of technologies that could make decisions that unfairly impact spe-
714 cific groups), privacy considerations, and security considerations.
715 • The conference expects that many papers will be foundational research and not tied
716 to particular applications, let alone deployments. However, if there is a direct path to
717 any negative applications, the authors should point it out. For example, it is legitimate
718 to point out that an improvement in the quality of generative models could be used to
719 generate Deepfakes for disinformation. On the other hand, it is not needed to point out
720 that a generic algorithm for optimizing neural networks could enable people to train
721 models that generate Deepfakes faster.
722 • The authors should consider possible harms that could arise when the technology is
723 being used as intended and functioning correctly, harms that could arise when the
724 technology is being used as intended but gives incorrect results, and harms following
725 from (intentional or unintentional) misuse of the technology.
726 • If there are negative societal impacts, the authors could also discuss possible mitiga-
727 tion strategies (e.g., gated release of models, providing defenses in addition to attacks,
728 mechanisms for monitoring misuse, mechanisms to monitor how a system learns from
729 feedback over time, improving the efficiency and accessibility of ML).

730 **11. Safeguards**

731 Question: Does the paper describe safeguards that have been put in place for responsible
732 release of data or models that have a high risk for misuse (e.g., pre-trained language models,
733 image generators, or scraped datasets)?

734 Answer: **[TODO]**

735 Justification: **[TODO]**

736 Guidelines:

- 737 • The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.